

PREDICTIVE ANALYTICS IN BANKING

A Comparative Study: XGBoost vs.
FT-Transformer Deep Learning for Customer
Churn Prediction

Comprehensive Project Report & Technical Architecture

Full-Stack Implementation & Cloud Deployment

Abstract and Executive Summary

In the modern financial sector, retaining customers is statistically proven to be vastly more cost-effective than acquiring new ones. Customer churn, the phenomenon where a client ceases their relationship with a bank, poses a severe threat to long-term profitability. This project presents a highly sophisticated, end-to-end Predictive Analytics platform engineered to accurately identify high-risk customers before they defect to competitors.

Unlike traditional data science projects that conclude at the Jupyter Notebook stage, this initiative bridges the gap between theoretical Machine Learning and production-grade Software Engineering. It introduces a comprehensive comparison between two vastly different algorithmic paradigms: **eXtreme Gradient Boosting (XGBoost)** and the cutting-edge **Feature Tokenizer + Transformer (FT-Transformer)**.

The system is wrapped in a high-performance Python FastAPI backend, containerized via Docker to ensure cross-platform compatibility, and deployed to a highly available Linux cloud environment. For end-users and business stakeholders, a stunning, glassmorphism-inspired React Single Page Application (SPA) was developed to visualize the model predictions in real-time. This report comprehensively documents the theoretical foundations, data preprocessing strategies, model architectures, and the complex DevOps pipeline required to bring this application to the global internet.

By leveraging this dual-model architecture, the bank's retention teams can instantly evaluate an individual's demographic and financial profile to receive not only a binary risk prediction but also a mathematically calculated confidence probability, enabling highly targeted and efficient marketing interventions.

Chapter 1: Introduction and Problem Statement

1.1 The Banking Ecosystem and Churn

The banking industry operates on incredibly tight margins, relying heavily on the lifetime value (LTV) of their customers. When a customer opens a checking account, takes out a mortgage, or utilizes a credit card, they generate consistent revenue over decades. However, the rise of digital-only "neobanks" and intense competition has made it easier than ever for consumers to switch institutions.

Customer churn is insidious; it often happens silently. A customer may slowly draw down their balance, stop using their credit card, and eventually close the account. By the time the bank realizes the customer is leaving, it is almost always too late to win them back. Traditional reactive methods (such as exit surveys) provide zero preventative value.

Proactive intervention is the only viable strategy. However, indiscriminately offering financial incentives (like waived fees or bonus interest rates) to every customer is financially ruinous. The bank must know exactly *who* is likely to leave so that retention budgets can be allocated exclusively to high-risk accounts.

1.2 The Role of Predictive Analytics

Predictive Analytics solves this problem by analyzing historical data to find hidden patterns indicative of future behavior. By analyzing thousands of past customers—both those who stayed and those who left—Machine Learning algorithms can learn the subtle behavioral footprints of a churner.

For example, human analysts might easily see that a customer with a very low balance is likely to leave. However, human analysts cannot easily compute how Age interacts with Geography, Tenure, and the Number of Products simultaneously across 10,000 customers. Machine Learning models map these high-dimensional relationships effortlessly.

1.3 Project Objectives

This project was initiated with several highly ambitious objectives that span Data Science, Backend Engineering, and Cloud Infrastructure:

- **Algorithmic Comparison:** To empirically test whether a traditional ensemble method (XGBoost) can be outperformed by a modern Deep Learning architecture (FT-Transformer) on structured tabular data.
- **API Development:** To engineer a robust REST API using FastAPI capable of loading multiple serialized models into RAM and performing highly concurrent inference.
- **Data Validation:** To strictly enforce incoming data schemas using Pydantic, ensuring that the machine learning models never crash due to malformed frontend requests.
- **User Interface:** To design a beautiful, responsive React frontend that abstracts away the complex mathematics and presents actionable business intelligence to non-technical users.
- **Cloud Deployment:** To conquer the inherent challenges of deploying heavy PyTorch tensors in memory-constrained cloud environments using Docker containerization.

The ultimate goal is to provide a fully functional product that a bank could immediately deploy to their internal intranet for use by their customer success teams.

Chapter 2: Data Exploration and Preprocessing

2.1 Dataset Overview

The foundation of any machine learning model is the quality of its data. This project utilizes a rich, anonymized dataset of 10,000 bank customers. The dataset contains 14 distinct columns, which were carefully analyzed during the Exploratory Data Analysis (EDA) phase to prevent data leakage.

Features such as RowNumber, CustomerId, and Surname were immediately dropped from the training set. While a neural network might accidentally find a spurious correlation between a customer's last name and their likelihood to churn, such features hold no true predictive power and would severely damage the model's ability to generalize to new customers.

2.2 Feature Analysis

The remaining features were categorized into continuous and categorical variables:

Categorical Features:

- **Geography:** The customer's country of residence (e.g., France, Spain, Germany). Cultural and regional economic differences deeply impact banking behavior.
- **Gender:** Male or Female.
- **HasCrCard:** A binary indicator of whether the customer holds a credit card with the bank.
- **IsActiveMember:** A binary indicator of recent account activity.

Categorical features cannot be fed directly into mathematical models; they must be encoded. In the XGBoost pipeline, techniques like One-Hot Encoding and Label Encoding were utilized to transform these strings into numerical matrices. In the FT-Transformer pipeline, the architecture natively tokenizes these categories into dense embeddings.

Continuous Features:

- **CreditScore:** A numerical representation of the customer's creditworthiness.
- **Age:** The customer's age in years. Older customers tend to have more stable finances and different retention patterns compared to younger, highly mobile demographics.
- **Tenure:** The number of years the customer has been with the bank.
- **Balance:** The amount of capital currently held in their accounts.
- **NumOfProducts:** The number of distinct financial products (loans, checking, savings) the customer utilizes.
- **EstimatedSalary:** The customer's reported annual income.

Continuous features required strict normalization. Algorithms rely on calculating distances and gradients; if `EstimatedSalary` ranges in the hundreds of thousands while `Tenure` ranges from 1 to 10, the algorithm will mathematically assume that `Salary` is vastly more important simply

because the numbers are larger. Standard Scaling ensures all features have a mean of 0 and a standard deviation of 1, allowing the models to evaluate them fairly.

2.3 Handling Imbalanced Data

A critical observation during the EDA phase was the severe imbalance in the target variable, `Exited`. Approximately 80% of the dataset consisted of customers who stayed, while only 20% represented churners.

If unaddressed, a machine learning model will become biased toward the majority class to minimize its overall error rate. To counteract this, we calculated the `scale_pos_weight` (the ratio of negative to positive samples) and injected it directly into the XGBoost loss function. This mathematically forces the algorithm to penalize mistakes made on the minority class (churners) much more heavily than mistakes made on the majority class.

Chapter 3: Traditional Machine Learning (XGBoost)

3.1 The Theory of Gradient Boosting

eXtreme Gradient Boosting (XGBoost) represents the pinnacle of traditional machine learning for tabular data. It is an ensemble learning method, meaning it does not rely on a single decision tree, but rather a "forest" of hundreds of trees working together.

Unlike Random Forests, which build trees independently and average their results, XGBoost builds trees *sequentially*. The first tree makes predictions, and inevitably makes errors. The second tree is then built specifically to predict the *errors* (residuals) of the first tree. The third tree predicts the errors of the second, and so on. This process of boosting allows the model to slowly converge on highly accurate, nuanced decision boundaries.

3.2 Hyperparameter Tuning with GridSearchCV

An untuned XGBoost model is highly prone to overfitting (memorizing the training data and failing on new data). To prevent this, we performed an exhaustive search over a massive hyperparameter grid using GridSearchCV.

The grid explored variations in:

- **n_estimators:** The total number of sequential trees (100, 200, 300).
- **max_depth:** The maximum depth of each tree (3, 4, 5). Shallow trees prevent the model from memorizing highly specific edge cases.
- **learning_rate:** The step size shrinkage (0.05, 0.1, 0.2). A smaller learning rate makes the model more robust by forcing it to learn slowly across more trees.

3.3 Cross-Validation Strategy

To ensure the hyperparameters chosen were truly optimal and not just lucky on a specific subset of data, we utilized 3-fold Cross-Validation. The training data was split into three equal chunks. The model was trained on two chunks and tested on the third, rotating until all chunks were used as testing data. The parameters that yielded the highest average score across all three folds were selected.

Crucially, the scoring metric used for the grid search was **F1-Score**, not Accuracy. Given the imbalanced nature of churn, maximizing the harmonic mean of Precision and Recall ensures the model does not achieve high scores by simply guessing "Stay" for every customer.

3.4 XGBoost Results and Feature Importances

After tuning, the XGBoost model achieved highly respectable metrics on the unseen test dataset. The model achieved a strong Recall, successfully identifying the vast majority of actual churners. However, this came at a slight cost to Precision, resulting in some False Positives.

The most fascinating outcome of the XGBoost training was the Feature Importance matrix. The algorithm mathematically ranked the features based on how often they were used to split the decision trees and how much they reduced the overall error. The top indicators of churn were:

1. **NumOfProducts:** Holding a specific number of products was overwhelmingly correlated with churn, suggesting potential dissatisfaction with specific product bundles.

2. **IsActiveMember:** Unsurprisingly, customers who rarely interacted with their accounts were highly prone to leaving.
3. **Age:** A strong non-linear relationship was found between specific age brackets and attrition rates.

Chapter 4: Deep Learning

(FT-Transformer)

4.1 The Tabular Data Challenge for Neural Networks

Deep Learning, specifically Neural Networks, has revolutionized Computer Vision (images) and Natural Language Processing (text). However, they have historically underperformed against XGBoost on tabular (spreadsheet) data. This is because tabular data lacks spatial or sequential correlation (the order of columns doesn't matter), and dense neural networks struggle with un-normalized categorical variables.

To overcome this, researchers developed the **Feature Tokenizer + Transformer (FT-Transformer)**, bringing the immense power of the Attention Mechanism (the exact same technology powering ChatGPT) to tabular datasets.

4.2 The FT-Transformer Architecture

The FT-Transformer solves the tabular problem in two distinct steps:

- **Feature Tokenization:** Every single feature—whether it is a continuous number like Balance or a categorical string like Geography—is passed through a dedicated embedding layer. This converts every cell in the dataset into a dense, multi-dimensional vector (a token).

- **Multi-Head Self-Attention:** These tokens are then fed into Transformer blocks. The Attention Mechanism allows the network to look at all features simultaneously and dynamically calculate how much "attention" one feature should pay to another. For example, it can learn that Age is highly relevant if Geography = Germany, but less relevant if Geography = Spain.

4.3 Implementation via PyTorch Tabular

Implementing a Transformer from scratch is mathematically complex and error-prone. This project utilized the `pytorch-tabular` framework to abstract the boilerplate code while maintaining deep control over the architecture.

The FT-Transformer was configured with strict parameters:

- **Task:** Classification
- **Attention Blocks:** 3 (Allowing the network to learn deep interactions)
- **Attention Heads:** 4 (Allowing the network to focus on 4 different feature interactions simultaneously)
- **Feed-Forward Layers:** 128-64-32 (Compressing the learned representations into a final prediction)
- **Activation Function:** ReLU (Rectified Linear Unit)

4.4 Training Dynamics

Because Transformers contain millions of parameters, training them on a standard CPU is prohibitively slow. The FT-Transformer was trained utilizing a GPU to accelerate matrix multiplications.

The training loop utilized an advanced AdamW optimizer and ran for 20 epochs with a batch size of 128. Once training completed, the final weights, biases, and optimizer states were serialized into a local directory (`ft_transformer_model`) so they could be effortlessly loaded by the API in production without needing a GPU.

Chapter 5: System Design and Full-Stack Engineering

5.1 The FastAPI Backend

A machine learning model is useless if it exists only in a Jupyter Notebook. It must be accessible. For this, we engineered a high-performance REST API using **FastAPI**.

FastAPI was selected for its native integration with Pydantic. We defined a strict schema (`ChurnPredictionRequest`) detailing the exact type and bounds of every expected feature. If a third-party application attempts to send invalid data, FastAPI instantly rejects the request with a 422 Unprocessable Entity error, guaranteeing that our PyTorch and XGBoost models are never subjected to malformed arrays.

Upon initialization, the server loads both the XGBoost `.json` weights and the PyTorch `.ckpt` directory into RAM. The `/predict` endpoint is fully asynchronous, parsing the incoming JSON, formatting it into a Pandas DataFrame, and routing it through both models concurrently.

5.2 The React + Vite Frontend

To provide a tangible interface for business users, a Single Page Application (SPA) was developed using React 18 and Vite. The UI was designed with a modern "glassmorphism" aesthetic, utilizing semi-transparent frosted glass elements over vibrant gradient

backgrounds.

The frontend features a comprehensive data-entry form. Upon submission, it triggers an asynchronous fetch request to the FastAPI backend. Crucially, the frontend is designed to dynamically parse the dual-payload response from the server, instantly rendering two side-by-side cards that visually contrast the predictions and confidence probabilities of both XGBoost and the FT-Transformer.

Chapter 6: Cloud Deployment and DevOps

6.1 Docker Containerization

Deploying complex Python environments (especially those involving PyTorch and XGBoost) directly to cloud servers is notoriously unstable due to operating system differences. To ensure absolute consistency, the entire backend was containerized using Docker.

The `Dockerfile` utilizes a lightweight `python:3.11-slim` image. All dependencies, model weights, and application scripts are packaged into a single, immutable artifact that can run identically on any machine in the world.

6.2 Conquering the PyTorch Memory Crisis

The most significant DevOps challenge encountered was cloud memory limits. When ``pip install torch`` is executed on a Linux server, it defaults to downloading the CUDA-enabled (GPU) version of PyTorch, which exceeds 3 Gigabytes. Free-tier cloud platforms (like Render) strictly limit RAM to 512MB.

This caused immediate "Out of Memory" (OOM) crashes during deployment. We successfully engineered a bypass by injecting a strict pre-installation command into the Dockerfile: `pip install --no-cache-dir torch --index-url`

<https://download.pytorch.org/whl/cpu>. This forced the server to download the lightweight CPU-only wheels, reducing the image footprint by over 80% and allowing flawless deployment.

6.3 Continuous Deployment and Latency Management

The Docker image was deployed as a Web Service on Render, while the React app was deployed as a Static Site via Render's global CDN. Environment variables (VITE_API_URL) were injected to seamlessly route frontend traffic to the backend API.

Because free-tier servers spin down after 15 minutes of inactivity (causing massive "cold-start" latency for the next user), we engineered a `/health` endpoint. We connected this endpoint to a global chronometer (cron-job.org) configured to ping the server every 14 minutes, mathematically guaranteeing the server never sleeps and providing zero-latency predictions 24/7.

Chapter 7: Final Results and Business Conclusion

7.1 Comparative Metrics

The ultimate test of this project was comparing the two models on the unseen hold-out dataset. Because churn is highly imbalanced, we prioritized the **F1-Score** and **ROC-AUC**.

XGBoost (Traditional ML) Performance:

- **Accuracy:** 74.0%
- **ROC-AUC Score:** 85.3%
- **Recall:** 72.2%
- **Precision:** 48.9%
- **F1-Score:** 58.3%

FT-Transformer (Deep Learning) Performance:

- **Accuracy:** 86.1%

- **ROC-AUC Score:** 90.2%

- **Recall:** 75.5%

- **Precision:** 62.7%

- **F1-Score:** 68.4%

7.2 Business Interpretation

While XGBoost proved to be a highly competent algorithm (achieving a strong Recall of 72.2%), it suffered from a high rate of False Positives, reflected in its low Precision (48.9%). In a business context, this means XGBoost would incorrectly flag many loyal customers as churners, leading the bank to waste marketing budgets offering them unnecessary retention discounts.

The **FT-Transformer vastly outperformed XGBoost**. By leveraging deep attention mechanisms to understand the non-linear interplay between variables, it not only increased the overall Recall (75.5%) but dramatically increased Precision (62.7%). This resulted in a vastly superior F1-Score (68.4%).

By utilizing the Deep Learning model, the bank can confidently allocate highly targeted retention campaigns to at-risk customers, maximizing ROI while minimizing wasted marketing spend on False Positives.

Appendix: Tech Stack and Version Requirements

The successful execution of this full-stack architecture relied on precise versioning of the following technologies:

Core Environment

- **Operating System:** Linux (Containerized via Docker)
- **Language:** Python 3.11, TypeScript

Machine Learning & Data Processing

- **XGBoost (v2.0.3):** Core traditional ML algorithm.
- **PyTorch (v2.14.0+cpu):** Deep learning framework (CPU optimized).
- **PyTorch Tabular (v1.2.0):** High-level API for the FT-Transformer.
- **Scikit-learn (v1.4.1):** Data scaling and metric evaluation.

Backend Infrastructure

- **FastAPI (v0.110.0):** Asynchronous API framework.
- **Uvicorn (v0.27.1):** ASGI web server implementation.

- **Pydantic:** Data validation schema engine.

Frontend Infrastructure

- **React 18:** Component-based UI library.
- **Vite (v8.3.0):** Next-generation frontend bundler.
- **CSS Modules:** Scoped, glassmorphism UI styling.